Aprendizaje Automático - Libro 1 - Clase 1 a 7

4. Clase 4

Clase Nº 4: Aprendizaje Supervisado

Contenido: Regresión lineal. Regresión logística.

En la clase de hoy trabajaremos los siguientes temas:

- Aprendizaje supervisado
- Regresión
- Creamos los conjuntos de datos
- Regresión contraída
- Clasificación

1. Presentación:

Bienvenidos a la clase N° 4 de Aprendizaje Automático, nos adentraremos en el mundo del Aprendizaje Supervisado, una de las piedras angulares de la inteligencia artificial. Exploraremos dos poderosos modelos matemáticos: la Regresión Lineal y la Regresión Logística. A través de estos modelos, aprenderemos cómo hacer predicciones y tomar decisiones basadas en datos. Además, descubriremos cómo implementar estas técnicas utilizando el popular lenguaje de programación Python.

Pero antes de sumergirnos en la construcción de modelos, primero nos sumergiremos en la preparación y organización de datos. Aprenderemos cómo analizar y limpiar datos para obtener información valiosa. Luego, abordaremos la creación de conjuntos de datos de entrenamiento y prueba, una práctica esencial para evaluar y validar nuestros modelos.

Prepárate para una emocionante aventura en el mundo del Aprendizaje Supervisado y la Regresión, donde las matemáticas se encuentran con la programación para dar vida a soluciones innovadoras y precisas en una amplia gama de aplicaciones. ¡Comencemos nuestro viaje hacia el Aprendizaje Automático!

En la clase pasada hemos trabajado sobre la visualización de datos con la librería Matplotlib.

Te invitamos ahora a visitar el siguiente recurso para introducirnos en el tema.

2. Desarrollo y Actividades:

Aprendizaje supervisado

Cuando trabajamos dentro de lo que es el aprendizaje supervisado, contamos con datos que poseen atributos adicionales, los cuales queremos predecir. Básicamente podemos contar con dos tipos de casos, que podemos nombrar como clasificación y regresión:

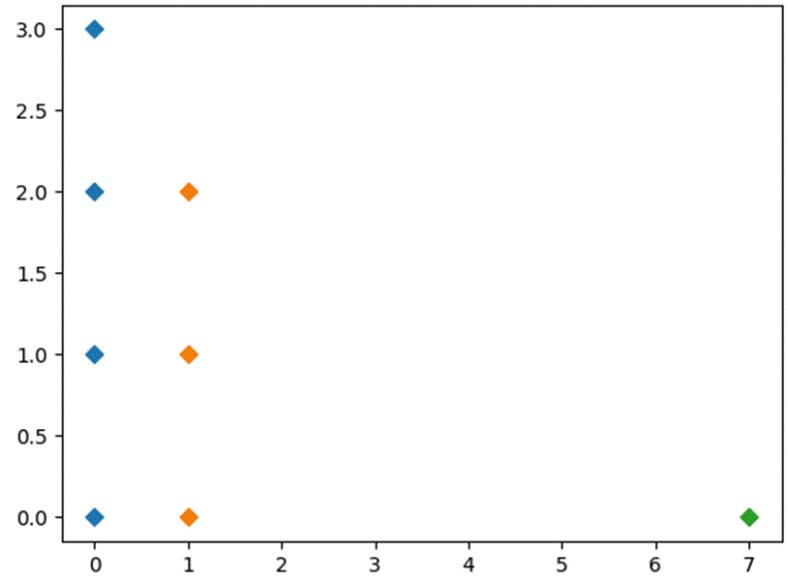
- *Regresión*: si la salida deseada consiste en una o más variables continuas, la tarea se llama regresión. Un ejemplo de un problema de regresión sería la predicción de la longitud de un salmón en función de su edad y peso.
- *Clasificación*: En este caso las muestras pertenecen a dos o más clases y queremos aprender de los datos ya etiquetados cómo predecir la clase de datos sin etiquetar. Un ejemplo de un problema de clasificación sería el reconocimiento de dígitos a mano, en el que el objetivo es asignar cada vector de entrada a una de un número finito de categorías discretas. Otra forma de pensar en la clasificación es como una forma discreta (en lugar de continua) de aprendizaje supervisado donde uno tiene un número limitado de categorías y para cada una de las *n* muestras proporcionadas, se trata de etiquetarlas con la categoría o clase correcta.

Uno de los puntos más difíciles en la creación de algoritmos es obtener datos que podamos utilizar confiadamente. El preprocesamiento de datos es un punto fundamental que debemos tomar muy en serio. Antes de utilizar los datos dentro de un algoritmo, debemos verificar la calidad y precisión de los mismos. En esta unidad comenzaremos a utilizar el análisis exploratorio de datos AED, el cual de caracteriza por su sencillez, debida principalmente a:

1. El empleo de gráficas y la reducción del uso de fórmulas algebraicas.
2. A que requiere muy pocas consideraciones previas sobre los datos.

El AED enfatiza la importancia de organizar y desplegar los datos gráficamente, de modo tal que se puedan percibir valores distintivos, como valores extremos, atípicos, conglomerados (grupos de datos muy cercanos entre sí) y brechas (separaciones de magnitudes considerables entre conglomerados), así como la identificación de patrones globales.

El AED nos lleva a plantearnos preguntas sobre los datos, por ejemplo tomemos la siguiente recopilación de datos a partir de números enteros:



Independientemente de lo que representan los datos, se puede apreciar en el diagrama que hay un valor considerablemente mayor que todos (7) que puede verse como atípico y que se tienen dos conglomerados, un grupo de cuatro valores en cero y un grupo de tres valores en uno.

Regresión - Regresión lineal

El método de regresión lineal consiste en identificar con precisión una línea que sea capaz de representar la distribución de puntos en un plano bidimensional. Dada la siguiente ecuación de la recta:

$$y = \frac{y_2 - y_1}{x_2 - x_1} * (x - x_1) + y_1 = \frac{y_2 - y_1}{x_2 - x_1} * x + \left(y_1 - \frac{y_2 - y_1}{x_2 - x_1} * x_1\right)$$

$$y = \alpha * x + \beta$$

El coeficiente alfa nos da la pendiente de la recta, mientras que el coeficiente beta nos da el punto de corte con el eje *y*.

Obtener los datos

Vamos a ver cómo realizar una regresión lineal a partir de los datos almacenados en el archivo *datos.csv*, importando los datos mediante *pandas* y convirtiendo las columnas en dos arrays de *numpy*.

```
import pandas as pd
import numpy as np
dat_csv  = pd.read_csv('datos/datos4.csv', encoding = "ISO-8859-1")
datos_x = dat_csv.x
datos_y = dat_csv.y
x = []
y = []
for i in dat_csv.x:
    x.append(i)
for j in dat_csv.y:
    y.append(j)
print(x)
print(y)

X = np.array(x)
Y = np.array(y)
```

Creamos los conjuntos de datos

Cuando construimos un modelo de aprendizaje automático, necesitamos una manera de validar nuestro modelo y verificar si está funcionando a un nivel satisfactorio. Para hacer esto, debemos separar nuestros datos en dos grupos: un *conjunto de datos de entrenamiento* y un *conjunto de datos de prueba*. El conjunto de datos de entrenamiento se usará para construir el modelo, y el conjunto de datos de prueba se usará para ver cómo este modelo entrenado se desempeña con datos desconocidos. Entonces, sigamos adelante y dividamos estos datos en los dos conjuntos:

```
# Calculo el 80 % de entrenamiento y 20 % prueba
datos_entrenamiento = int(0.8 * len(X))
datos_prueba = len(X) - datos_entrenamiento
print(datos_entrenamiento)
print(datos_prueba)
```

```
#Datos de entrenamiento
X_entrenamiento = X[:datos_entrenamiento].reshape((datos_entrenamiento,1))
Y_entrenamiento = Y[:datos_entrenamiento].reshape((datos_entrenamiento,1))
print(X_entrenamiento)
print(Y_entrenamiento)
```

```
#Datos de prueba
X_prueba = X[datos_entrenamiento:].reshape((datos_prueba,1))
Y_prueba = Y[datos_entrenamiento:].reshape((datos_prueba,1))
print(X_prueba)
print(Y_prueba)
```

Realizamos la regresión lineal

Para realizar la regresión lineal, haremos uso del objeto *linear_model.Regression()* de *scikit-learn*, el cual hace uso del método *fit()* para tomar los dos datos a ajustar y obtener la regresión, luego a partir de los datos de regresión encontrados podemos observar la curva de ajuste de entrenamiento junto con la dispersión de datos de entrenamiento.

```
from sklearn import linear_model

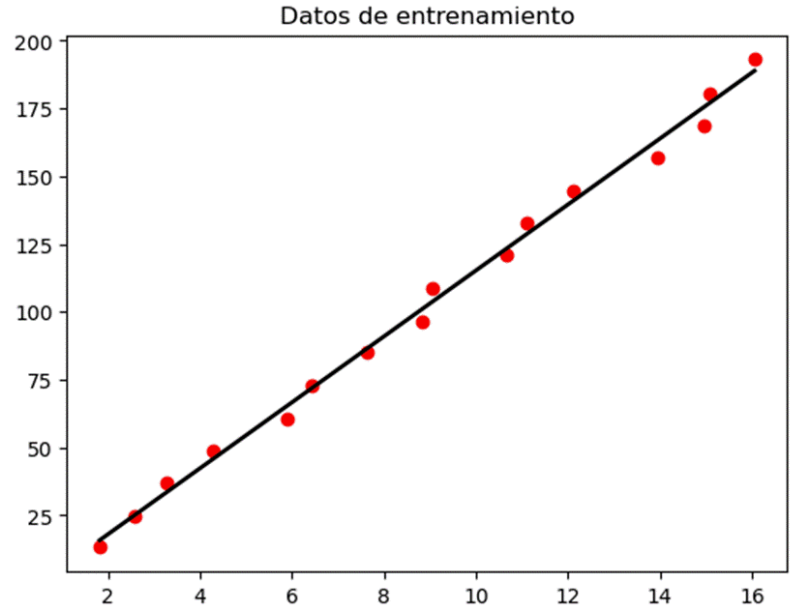
# Creamos un objeto de regresión lineal
linear_regressor = linear_model.LinearRegression()

# Ajustamos la regresión a los datos de entrenamiento
linear_regressor.fit(X_entrenamiento, Y_entrenamiento)

# Y a partir de datos de regresión encontrados
Y_predicha_de_entrenamiento = linear_regressor.predict(X_entrenamiento)

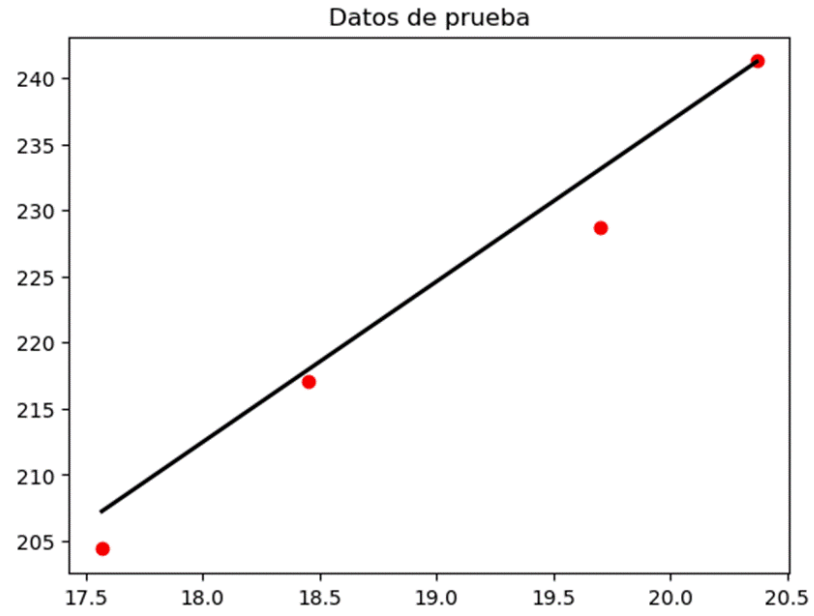
import matplotlib.pyplot as plt

plt.figure()
plt.scatter(X_entrenamiento, Y_entrenamiento, color='red')
plt.plot(X_entrenamiento, Y_predicha_de_entrenamiento, color='black', linewidth=2)
plt.title('Datos de entrenamiento')
plt.show
```



Una vez que visualizamos el ajuste podemos evaluar la correspondencia de la regresión con relación a los datos de prueba.

```
Y_predicha_de_prueba = linear_regressor.predict(X_prueba)
plt.figure()
plt.scatter(X_prueba, Y_prueba, color='red')
plt.plot(X_prueba, Y_predicha_de_prueba, color='black', linewidth=2)
plt.title('Datos de prueba')
plt.show
```



Existen varias formas de evaluar una regresión lineal, y podemos utilizar nuevamente *scikit-learn* para realizar esta tarea, mediante el módulo *metrics* según se muestra a continuación:

```
import sklearn.metrics as sm
print("Error absoluto medio =", round(sm.mean_absolute_error(Y_prueba, Y_predicha_de_prueba), 2))
print("Error cuadratico medio =", round(sm.mean_squared_error(Y_prueba, Y_predicha_de_prueba), 2))
print("Error absoluto mediano =", round(sm.median_absolute_error(Y_prueba, Y_predicha_de_prueba), 2))
print("Puntuación de varianza explicada =", round(sm.explained_variance_score(Y_prueba, Y_predicha_de_prueba), 2))
print("Puntuación R2 =", round(sm.r2_score(Y_prueba, Y_predicha_de_prueba), 2))
```

- **Error absoluto medio**: este es el promedio de errores absolutos de todos los puntos de datos en el conjunto de datos dado.
- **Error cuadrático medio**: Este es el promedio de los cuadrados de los errores de todos los puntos de datos en el conjunto de datos dado. ¡Es una de las métricas más populares que hay!
- **Error absoluto mediana**: Esta es la mediana de todos los errores en el conjunto de datos dado. La principal ventaja de esta métrica es que es robusta a los valores atípicos. Un único punto negativo en el conjunto de datos de prueba nodistorsionara la métrica de error completa, en lugar de una métrica de error promedio.
- **Puntuación de varianza explicada**: esta puntuación mide qué tan bien nuestro modelo puede explicar la variación en nuestro conjunto de datos. Una puntuación de 1.0 indica que nuestro modelo es perfecto.
- **Puntuación R2**: Se nombra como R cuadrado, y esta puntuación se refiere al coeficiente de determinación. Esto nos dice qué tan bien serán predichas las muestras desconocidas por nuestro modelo. La mejor puntuación posible es 1.0, pero la puntuación también puede ser negativa.

Los valores obtenidos son:

```
Error absoluto medio = 2.02
Error cuadratico medio = 7.01
Error absoluto mediano = 1.83
Puntuación de varianza explicada = 0.98
Puntuación R2 = 0.96
```

Regresión contraída

En ocasiones cuando vamos a realizar una regresión lineal, nos encontramos que pueden existir algunos valores aislados, quizás producto de una mala toma de datos que se alejan de la dispersión y modifican el valor de los coeficiente encontrados. Cuando realizamos la regresión lineal por el método de mínimos cuadrados, los coeficientes se estiman determinando valores numéricos que minimizan la suma de las desviaciones al cuadrado entre las respuestas observadas y las respuestas ajustadas, de acuerdo con la siguiente ecuación:

$$RSS = \sum_{i=1}^{n} \quad (y_i - \beta_1 * x_i + \beta_2)^2$$

La regresión ridge o regresión contraída, para estimar los coeficientes β, comienza a partir de la fórmula básica de la suma residual de cuadrados (RSS) y agrega el término de penalización. λ (≥ 0) el cual se define como el parámetro de ajuste, que se multiplica por la suma de los coeficientes de β al cuadrado (excluyendo la intersección) para definir el período de penalización, como se muestra en la siguiente ecuación:

$$RSS + \lambda * \beta_1^2 = \sum_{i=1}^{n} (y_i - \beta_1 * x_i + \beta_2)^2 + \lambda * \beta_1^2$$

```
from sklearn import linear_model
regresion_ridge = linear_model.Ridge(alpha=0.01, fit_intercept=True, max_iter=10000)
regresion_ridge.fit(X_prueba, Y_prueba)

Y_predicha_de_prueba = regresion_ridge.predict(X_prueba)


print("Error absoluto medio =", round(sm.mean_absolute_error(Y_prueba, Y_predicha_de_prueba), 2))
print("Error cuadratico medio =", round(sm.mean_squared_error(Y_prueba, Y_predicha_de_prueba), 2))
print("Error absoluto mediano =", round(sm.median_absolute_error(Y_prueba, Y_predicha_de_prueba), 2))
print("Puntuación de varianza explicada =", round(sm.explained_variance_score(Y_prueba, Y_predicha_de_prueba), 2))
print("Puntuación R2 =", round(sm.r2_score(Y_prueba, Y_predicha_de_prueba), 2))

plt.figure()
plt.scatter(X_prueba, Y_prueba, color='red')
plt.plot(X_prueba, Y_predicha_de_prueba, color='black', linewidth=2)
plt.title('Datos de prueba')
plt.show
```

En este caso los valores son:

```
Error absoluto medio = 1.49
Error cuadratico medio = 2.86
Error absoluto mediano = 1.49
Puntuación de varianza explicada = 0.98
Puntuación R2 = 0.98
```

Actividad 1

Es importante que realices todas las actividades, estas te permitirán saber como vas avanzando y podrás reflexionar sobre la teoría si te queda alguna duda de cómo hacer esta actividad por favor consultanos en el campus.

El objetivo de esta actividad es encontrar un dataset de tu interés que contenga como variable objetivo una variable continua y aplicar la regresión lineal para intentar predecirla a partir de las demás variables predictoras.

El método que seguiremos es el siguiente:

- Preparación y organización de datos
- Exploración de los datos
- Modelado de datos

Ahora seguimos adelante con un poco más de teoría que nos ayudará a andar en el tema.

Clasificación - Regresión logística

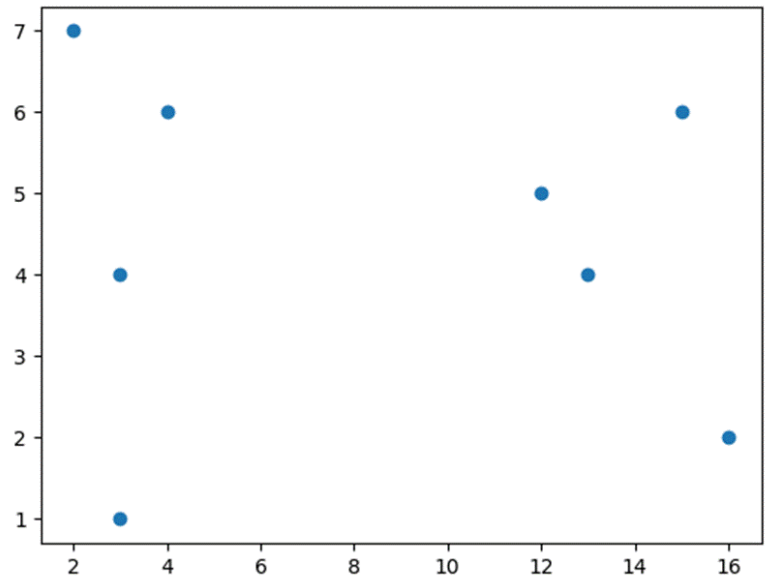
La regresión logística (Logistic Regression), a pesar de su nombre, es un modelo lineal para la clasificación en lugar de la regresión. La regresión logística también se conoce en la literatura como regresión logit, clasificación de máxima entropía (MaxEnt) o el clasificador log-lineal. En este modelo, las probabilidades que describen los posibles resultados de un solo ensayo se modelan utilizando una función logística.

Veamos cómo aplicar la regresión logística aplicando el proceso a la clasificación de una serie de puntos, es un caso abstracto para generalizar el procedimiento, pero podríamos aplicarlo a algo más concreto como la clasificación de mails para determinar si es spam o no.

Comencemos por tomar una serie de puntos según se muestra a continuación y graficarlos como hemos aprendido a hacer hasta aquí.

```
import numpy as np
from sklearn import linear_model
from sklearn import model_selection
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

X = np.array([[16,2], [3,1], [2,7], [13,4], [3,4], [12,5], [15,6], [4,6]])
plt.figure()
plt.scatter(X[:,0],X[:,1])
plt.show()
```



Dados los puntos de dispersión parecen existir dos grupos de datos, uno que se agrupa en torno al valor x = 3 y otro que se agrupa en torno al valor x = 14, lo que haremos es en base a esta hipótesis asignar a los valores cercanos a 3 el valor de cero y a los cercanos a catorce el valor de uno y agregaremos los valores en el mismo orden dentro del array Y.

```
Y = [1, 0, 0, 1, 0, 1, 1, 0]
```

Ahora utilizaremos el objeto clasificador *LogisticRegresion()* y le pasaremos los datos que deseamos clasificar de forma de crear un modelo, haciendo que se ajuste mediante el uso de *fit()*

```
clasificador = linear_model.LogisticRegression(solver='lbfgs', C=100)
clasificador.fit(X, Y)
```

Con la clasificación realizada, utilizaremos el método *predict()* de forma de predecir como el modelo predice los propios valores pasados como datos. Este es un caso muy simple en el cual la predicción retorna exactamente los mismos valores, sin embargo en otros casos más complejos el ajuste no es del 100% y el modelo debe ser ajustado a medida que conocemos más la problemática del tema tratado.

```
prediccion = clasificador.predict(X)

print(prediccion)
```

```
[1 0 0 1 0 1 1 0]
```

```
print(clasificador.score(X,Y))
```

```
1.0
```

Ahora en base a otros datos recopilados podemos ver como el modelo predice al grupo al cual deberían pertenecer los datos.

```
Xn = np.array([[6,4], [20,7], [4,17]])
Yn = clasificador.predict(Xn)

print(Yn)
```

```
[0 1 0]
```

Siempre es mejor tratar de visualizar los datos para tener una mejor comprensión del tema, por lo que vamos a graficar los datos asignando como colores las opciones de ceros o unos de la clasificación (c=Y).

Tomaremos los datos que utilizamos para crear el modelo y los que hemos utilizado para testear el modelo y los concatenamos para formar un único array en x e y, y luego determinaremos el rango de visualización en el cual veremos los datos.

```
Xs = np.concatenate((X, Xn), axis=0)
Ys = np.append(Y , Yn)

x_min, x_max = min(Xs[:, 0]) - 1.0, max(Xs[:, 0]) + 1.0
y_min, y_max = min(Xs[:, 1]) - 1.0, max(Xs[:, 1]) + 1.0
```

Dado ahora los puntos de los ejes x e y crearemos a partir de ellos una malla de puntos lo cual es en la práctica una cuadrícula rectangular creada a partir de los datos.

```
step_size = 0.01

valor_x, valor_y = np.meshgrid(np.arange(x_min, x_max, step_size), np.arange(y_min, y_max, step_size))

malla_de_puntos = clasificador.predict(np.c_[valor_x.ravel(), valor_y.ravel()])
```

Aquí estamos utilizando el método *ravel()* de forma de crear una copia de los valores originales. También podríamos haber utilizado el método *flatten()*, la diferencia entre ambos son:

ravel (): Retorna una referencia a la matriz original, si se modifica la matriz, el valor de la matriz original también cambia. *ravel()* es más rápido que *flatten()*.

flatten(): Retorna una copia de la matriz original, si se modifica algún valor de esta matriz, el valor de la matriz original no se verá afectado.

También notar que estamos utilizando *np.c_*, para obtener los datos por pares x, y ya que dada una matriz nos retorna sus valores por columnas.

Creamos un mapa de color a partir de los datos anteriores mediante *pcolormesh()*

```
malla_de_puntos = malla_de_puntos.reshape(valor_x.shape)
plt.figure()
plt.pcolormesh(valor_x, valor_y, malla_de_puntos, cmap=plt.cm.gray, shading='auto')
```

Para comprender cómo funciona *reshape()* consideremos que tenemos el siguiente ejemplo en el cual tenemos dos arrays

```
x = np.array([1, 5, 10])
y = np.array([1, 4, 8])
z = np.linspace(0, 255, 9).reshape(3, 3)
print(z)
plt.pcolormesh(x, y, z, cmap = "PuRd", shading ='auto'); #cmap = "PuRd");
plt.colorbar();
```

pcolormesh() crea una gráfica en pseudo-colores con un grid no rectangular. p*colormesh([x, y,] c, **kwargs)*

Aquí:

- c es un array 2D de escalares, valores que serán "mapeados" a un conjunto de colores determinado por el argumento cmap,
- x e y, son argumentos opcionales, definen las coordenadas de las esquinas de los rectángulos que se mostrarán del color correspondiente según se muestra a continuación.

```
(X[i+1, j], Y[i+1, j])         (X[i+1, j+1], Y[i+1, j+1])
                       +--------+
                       | C[i,j] |
                       +--------+
    (X[i, j], Y[i, j])         (X[i, j+1], Y[i, j+1]),
```

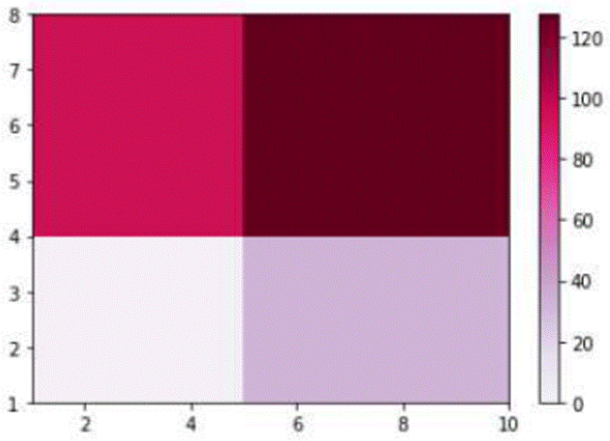
fuente: https://scikit-learn.org/stable/

De hecho, los "rectángulos" en cuestión están limitados por los valores de *x* e *y*. En el este ejemplo los valores de *x* 1, 5 y 10 definen dos rectángulos, uno que va a tener como límites en una dimensión los valores 1 y 5, y otro que va a tener como límites los valores 3 y 10.

Los valores de *x* y de *y* determinan los límites de los rectángulos, lo que significa que el array *z* no necesita tener más que *x*-1 elementos horizontales e *y*-1 elementos verticales. O dicho con otras palabras, de nuestro array *z* de tamaño 3x3 se está ignorando una fila y una columna como se muestra a continuación.

```
[[  0.     31.875  63.75 ]
 [ 95.625 127.5   159.375]
 [191.25  223.125 255.    ]]
```

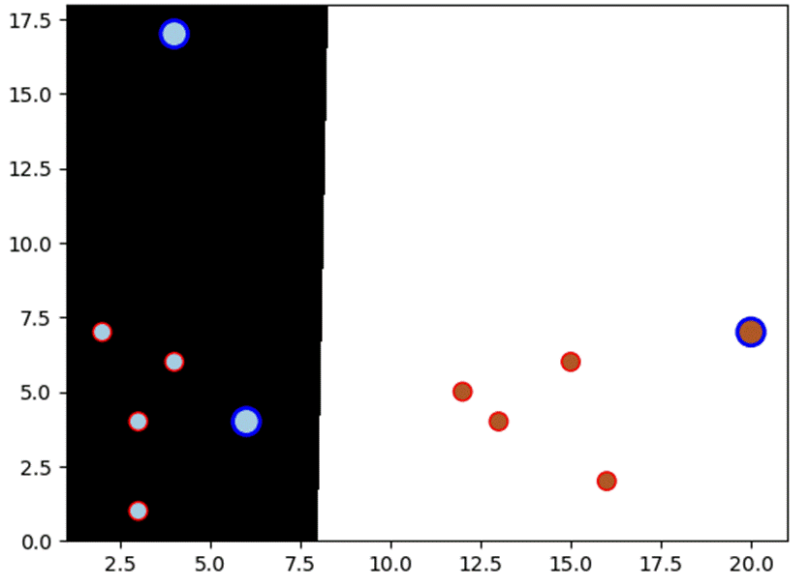
Al graficar los datos anteriores nos quedan los cuatro rectángulos:



Finalmente podemos poner sobre un gráfico todos los puntos tanto los de entrenamiento como los de testeo y comprobar que el modelo a separar correctamente los valores:

```
malla_de_puntos = malla_de_puntos.reshape(valor_x.shape)
plt.figure()
plt.pcolormesh(valor_x, valor_y, malla_de_puntos, cmap=plt.cm.gray, shading='auto')
plt.scatter(X[:, 0], X[:, 1], c=Y, s=80, edgecolors='red', linewidth=1, cmap=plt.cm.Paired)
plt.scatter(Xn[:, 0], Xn[:, 1], c=clasificador.predict(Xn), s=180, edgecolors='blue', linewidth=2,
            cmap=plt.cm.Paired)

plt.show()
```



Actividad 2

Te proponemos entonces que realices ahora la siguiente actividad con los conceptos leídos anteriormente.

Para esta actividad se ha creado un archivo *usuarios_win_mac_lin.csv* con datos de entrada a modo de ejemplo para clasificar si el usuario que visita un sitio web usa como sistema operativo Windows, Macintosh o Linux.

Nuestra información de entrada son 4 características que se tomó de una web que utiliza Google Analytics y son:

- Duración de la visita en Segundos
- Cantidad de Páginas Vistas durante la Sesión
- Cantidad de Acciones del usuario (click, scroll, uso de checkbox, sliders,etc)
- Suma del Valor de las acciones (cada acción lleva asociada una valoración de importancia)

Como la salida es discreta, asignaremos los siguientes valores a las etiquetas:

0 – Windows

1 – Macintosh

2 -Linux

Tu tarea es crear una Jupyter Notebook *Regresion_logistica.ipynb* que permita predecir qué sistema operativo utiliza un usuario según los datos de entrada.

¿Cómo pudiste llevar adelante esta actividad? te pareció muy difícil o muy fácil? ¿por qué? Por favor compartí tu respuesta en el foro del Campus

3. Instancia de Evaluación Espacio de entrega de tarea:

En esta instancia de evaluación te pedimos que puedas poner en juego lo que has aprendido hasta este momento. Recordá que siempre podrás volver a la teoría presentada en esta clase o hacer las preguntas pertinentes en los encuentros sincrónicos o las tutorías presenciales.

Es importante tener en cuenta que para aprobar esta instancia deberás:

- Entregar en tiempo y forma
- Realizar un 60% de la actividad correctamente.
- Cumplir con las consignas
- Cumplir con el formato requerido.

La nota mínima para aprobar la tarea es 6.

Deberás entregar las actividades que se proponen en esta clase en tu repositorio GITHUB.

4. Cierre:

Es fundamental adquirir un sólido entendimiento de modelos como la Regresión Lineal y la Regresión Logística, así como su implementación en Python, en el contexto del Aprendizaje Automático. Estos modelos actúan como pilares fundamentales al brindar las bases para comprender conceptos más avanzados. Además, dominar la preparación, organización y visualización de datos, sienta las bases para el éxito en proyectos de Aprendizaje Automático. Esta capacidad no solo permite construir modelos precisos, sino también evaluar su desempeño de manera efectiva, lo que es crucial para tomar decisiones informadas y aplicar el Aprendizaje Automático de manera efectiva en una variedad de campos, desde la medicina hasta el análisis financiero y la inteligencia artificial. En última instancia, este conocimiento es esencial para convertirse en un profesional competente y contribuir al avance de esta emocionante disciplina.

Te invitamos a que nos dejes tus impresiones sobre lo que aprendimos hoy en el foro. También podés compartir links de artículos o videos que te hayan parecido interesantes mientras estudiabas este tema.

Te esperamos en los próximos encuentros. Recordá que es importante participar de los foros, de los encuentros sincrónicos y en lo posible de las tutorías presenciales.

Para finalizar te dejamos estos recursos que pueden ayudarte a entender el tema un poco más.

Sitios

- Martinez, Jose. “Regresión Lineal: teoría y ejemplos en Python.” IArtificial.net, 2 October 2020, https://www.iartificial.net/regresion-lineal-con-ejemplos-en-python/. Accessed 9 September 2023.
- Martinez, Jose. “Regresión Logística para Clasificación.” IArtificial.net, 21 September 2020, https://www.iartificial.net/regresion-logistica-para-clasificacion/. Accessed 9 September 2023.

4. Bibliografía

OBLIGATORIA

- HAN, R. (2019): *Matemáticas del Aprendizaje Automático. Introducción a la analítica de datos e Inteligencia Artificial.*
- JONES, H. (2019): *Aprendizaje Automático. El Aprendizaje Automático para principiantes que desean comprender aplicaciones, Inteligencia Artificial, Minería de Datos, Big Data y más*. Editorial Edición Kindle.
- RUSSELL, R. (2018): *Machine Learning. Guia Paso a Paso Para Implementar Algoritmos De Machine Learning Con Python*
- VIDALES, A. (2019) *Machine Learning Con Matlab*. Editorial Edición Kindle.
- CANE, A. (2019): *Programación Con Python. Guía completa para principiantes.*
- HERNÁNDEZ, V. (2019): *Aprendizaje Automático. Implementación en Inteligencia Artificial.* Editorial Edición Kindle.

SUGERIDA

- “Introduction to TensorFlow.” *TensorFlow*, https://www.tensorflow.org/learn. Accedido el 23 Junio del 2023.
- “IPython.” *Jupyter and the future of IPython — IPython*, http://ipython.org/. Accedido el 23 Junio del 2023.
- “Matplotlib.” *Matplotlib — Visualization with Python*, https://matplotlib.org/. Accedido el 23 Junio del 2023.
- “NumPy.org.” *NumPy.org*, http://www.numpy.org/. Accedido el 23 Junio del 2023.
- “pandas.” *pandas - Python Data Analysis Library*, https://pandas.pydata.org/. Accedido el 23 Junio del 2023.
- “Requests: HTTP para Humanos.” *Requests: HTTP para Humanos — documentación de Requests - 1.1.0*, http://es.python-requests.org/es/latest/. Accedido el 23 Junio del 2023.
- “scikit-learn.” *scikit-learn: machine learning in Python — scikit-learn 1.2.2 documentation*, https://scikit-learn.org/stable/. Accedido el 23 Junio del 2023.
- “seaborn.” *seaborn: statistical data visualization — seaborn 0.12.2 documentation*, https://seaborn.pydata.org/. Accedido el 23 Junio del 2023.